



Facultad: Ciencias Aplicadas
Escuela: Ciencias de la computación
Tutor: Darío Sebastián Cabezas Erazo
Período: Marzo2026–Junio2026
Actividad: Midterm - Practical
Nombre Estudiante: Santiago Vallejo

1. Introduction

In the following project, an authentication API was developed using FastAPI, SQLAlchemy, and SQLite, with the objective of implementing a basic identity management system by applying modern security measures for credential protection. During the development process, user registration and login functionalities were implemented, incorporating hashing techniques through bcrypt to prevent passwords from being stored in plain text. In addition, security concepts such as the automatic Salt provided by bcrypt and an additional Pepper defined within the application were applied in order to strengthen protection against attacks targeting compromised databases. Authentication tests and verification of the data stored in SQLite were also carried out to confirm that the passwords were correctly encrypted and that different users using the same password generated different hashes.

The development of this practice made it possible to understand the importance of properly protecting user credentials, as well as the operation of different mechanisms currently used in real authentication and cybersecurity systems.

2. Objective

Develop a robust identity management system using FastAPI and SQLAlchemy. The student must apply advanced security layers to protect user credentials, understanding the technical implementation of modern hashing algorithms and the critical difference between adding a Salt (user-specific randomness) and a Pepper (global server secret).

3. Theoretical Framework

a) Technical comparison: ¿Why should fast algorithms such as MD5 or SHA-256 not be used for passwords?

Algorithms such as MD5 or SHA-256 were mainly designed to generate hashes quickly. This is useful for verifying the integrity of files or information, but this speed becomes a major problem when they are used for password storage, since an attacker can test millions of combinations per second using brute force attacks or password dictionaries, making it easier to discover the password and gain access to the information.

For this reason, it is recommended to use slow hashing algorithms such as bcrypt or PBKDF2, since they were specifically designed to protect passwords. In this project, bcrypt was used because it is simple to implement and automatically adds a unique salt for each user. This allows completely different hashes to be generated even when users have the same password.

b) Explanation of the data flow: At what point is the Pepper added and why is it not stored in the database?

The Pepper was added before performing the password hashing process. First, the password entered by the user is concatenated with a secret string defined within the server, and then the result is sent to the bcrypt algorithm to generate the final hash.

The Pepper should not be stored in the database because its function is to act as an additional security layer independent from the stored data. If an attacker manages to obtain the database file but does not know the Pepper used on the server, it becomes much more difficult to discover the original passwords even if they have access to the hashes, therefore providing stronger protection for the information.

4. Development and Evidence

- **Configuration of models with SQLAlchemy**

Here, the main user model was created using SQLAlchemy. It contains an id, a unique username, and the field where encrypted passwords generated through hashing will be stored.

```
class User(SQLModel, table=True):  
    id: int | None = Field(default=None, primary_key=True)  
    username: str = Field(unique=True)  
    hashed_password: str
```

- **Hashing functions and Pepper integration**

Functions were implemented to generate and verify hashes using bcrypt. Before generating the hash, the password is concatenated with the Pepper defined in the application, adding an extra layer of security.

```
def hash_password(password: str) -> str:
    peppered_password = password + PEPPER

    hashed = bcrypt.hashpw(
        peppered_password.encode("utf-8"),
        bcrypt.gensalt()
    )

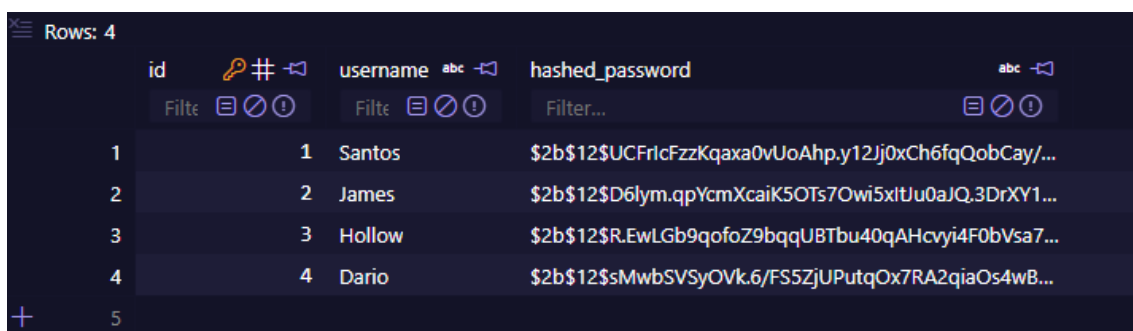
    return hashed.decode("utf-8")

def verify_password(password: str, hashed_password: str) -> bool:
    peppered_password = password + PEPPER

    return bcrypt.checkpw(
        peppered_password.encode("utf-8"),
        hashed_password.encode("utf-8")
    )
```

- **Evidence from the database**

When reviewing the SQLite database, it can be observed that passwords are not stored in plain text. Instead, the hashes generated by bcrypt are displayed. It is also verified that users with the same password produce different hashes thanks to the automatic use of Salt.



id	username	hashed_password
1	Santos	\$2b\$12\$UCFrlcFzzKqaxa0vUoAhp.y12Jj0xCh6fqQobCay/...
2	James	\$2b\$12\$D6lym.qpYcmXcaiK5OTs7Owi5xltUu0aJQ.3DrXY1...
3	Hollow	\$2b\$12\$R.EwLGb9qofoZ9bqqUBTbu40qAHcvyi4F0bVsa7...
4	Dario	\$2b\$12\$sMwbSVSyOVk.6/FS5ZjUPutqOx7RA2qiaOs4wB...

5. Analysis and Reflection

What impact does the Pepper have on security if the attacker manages to download the .db file but does not have access to the server's environment variables?

The impact of the Pepper is significant because it adds an additional layer of protection since it is not stored together with the database data. If an attacker obtains only the .db file, they would still need to know the exact Pepper used on the server in order to correctly verify the passwords. This increases the difficulty of brute force or dictionary attacks, because having only the hashes would not be enough for the attacker to crack the passwords without knowing the additional secret.

What is the “cost” or “iterations” in the chosen algorithm and how does it help prevent brute force attacks?

The cost represents the amount of work that the algorithm must perform to generate a hash with bcrypt. This value determines how many times the password is internally processed before obtaining the final result. The higher the cost, the more time it takes to generate and verify each hash. This helps prevent brute force attacks because it significantly reduces the speed at which an attacker can automatically test passwords.

6. Bibliography

Freda, A. (2022, January 7). *¿Qué es el algoritmo de hashing MD5 y cómo funciona? ¿Qué Es El Algoritmo de Hashing MD5 Y Cómo Funciona?; Avast.*
<https://www.avast.com/es-es/c-md5-hashing-algorithm>

Wagner, L. (2020, July 8). *What is SHA-256? Boot.Dev Blog; Boot.dev.*
<https://www-boot-dev.translate.goog/blog/computer-science/how-sha-2-works-step-by-step-sha-256/? x tr sl=en& x tr tl=es& x tr hl=es& x tr pto=tc>

<https://www.facebook.com/littlefiredigital>. (2022, February 23). *Little Fire Digital Ltd. Little Fire Digital Ltd.* <https://little-fire.com/salt-and-pepper-in-password-cryptography/>